Tutorial 3 - I/O Ports and Device Interfacing (Part 2)

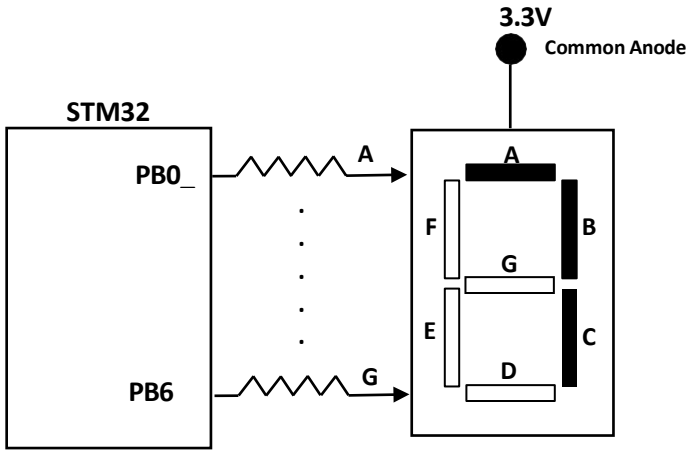
Q3-2.1 A **common cathode** 7-segment display device is connected to Port B (segment "A" to PB_0, segment "B" to PB_1... decimal point to PB_7).

Which of the following lines of C code will result in a digit "7" displayed on the 7-Segment display?

(a) PortOut Seg1(PortB, 0x000000FF);
Seg1 = 0x07;
(b) PortIn Seg1(PortB, 0x000000FF);
Seg1 = 0x07;
(c) PortOut Seg1(PortB, 0x00000007);
Seg1 = 0xF0;
(d) PortOut Seg1(PortB, 0x00000007);

d

Q3-2.2 Based on the circuit diagram below (decimal point to PB7), what is the correct output value to Port B (PB7:PB0) to display the digit "7"?



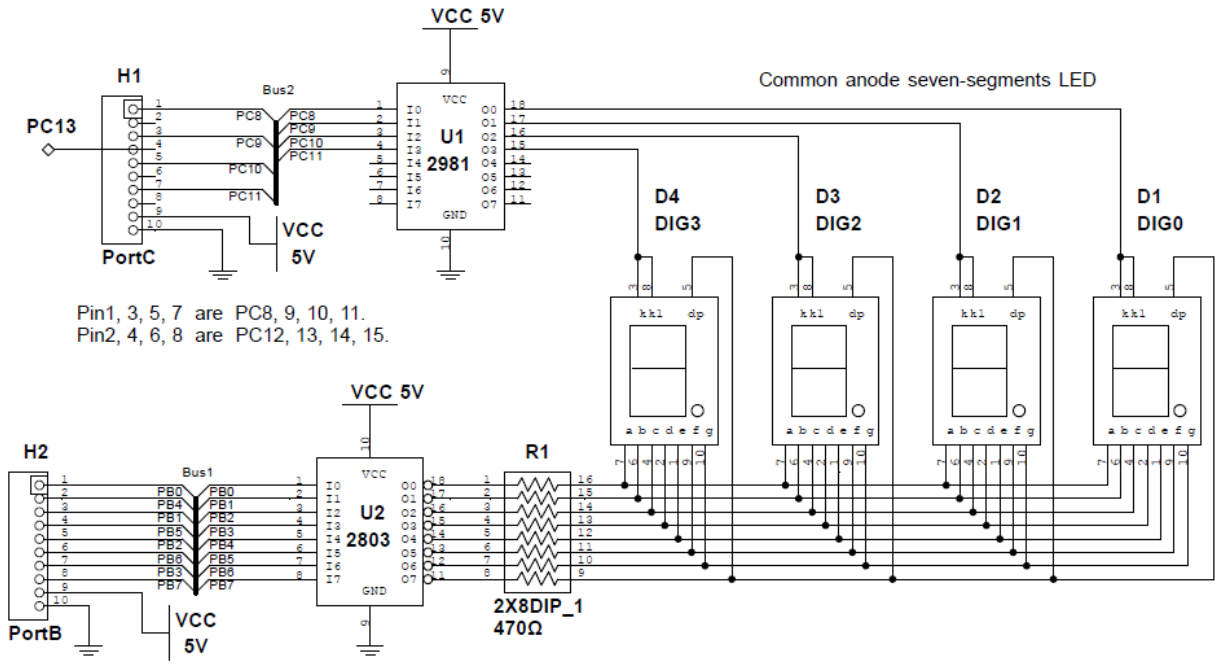
(a) 0xF0
(b) 0xF8
(c) 0x02
(d) 0x11

b

Q3-2.3 Based on the circuit diagram below (MAPP experiment board),

(a) what is the correct output value to Port B (PB7:PB0) to display the digit '7'?

(b) derive all the output value to Port B (PB7:PB0) to display the digit from '0' to 'F'.



a) 0xF8, 0b11111000

b)
0: 0xC0
1: 0xF9
2: 0xA4
3: 0xB0
4: 0x99
5: 0x92
6: 0x82
7: 0xF8
8: 0x80
9: 0x90
A: 0x88
B: 0x83
C: 0xC6
D: 0xA1
E: 0x86
F: 0x8E

Q3-2.4 The STM32F103 Port configuration register (Low & High) is shown below.

Port configuration register low (GPIOx_CRL) (x=A..G)

Address offset: 0x00

Reset value: 0x4444 4444

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CNF7[1:0]		MODE7[1:0]		CNF6[1:0]		MODE6[1:0]		CNF5[1:0]		MODE5[1:0]		CNF4[1:0]		MODE4[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNF3[1:0]		MODE3[1:0]		CNF2[1:0]		MODE2[1:0]		CNF1[1:0]		MODE1[1:0]		CNF0[1:0]		MODE0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:30, 27:26, 23:22, 19:18, 15:14, 11:10, 7:6, 3:2 **CNFy[1:0]:** Port x configuration bits (y= 0 .. 7)
These bits are written by software to configure the corresponding I/O port.
Refer to *Table 20: Port bit configuration table*.
In input mode (MODE[1:0]=00):
00: Analog mode
01: Floating input (reset state)
10: Input with pull-up / pull-down
11: Reserved
In output mode (MODE[1:0] > 00):
00: General purpose output push-pull
01: General purpose output Open-drain
10: Alternate function output Push-pull
11: Alternate function output Open-drain

Bits 29:28, 25:24, 21:20, 17:16, 13:12, 9:8, 5:4, 1:0 **MODEy[1:0]:** Port x mode bits (y= 0 .. 7)
These bits are written by software to configure the corresponding I/O port.
Refer to *Table 20: Port bit configuration table*.
00: Input mode (reset state)
01: Output mode, max speed 10 MHz.
10: Output mode, max speed 2 MHz.
11: Output mode, max speed 50 MHz.

Port configuration register high (GPIOx_CRH) (x=A..G)

Address offset: 0x04

Reset value: 0x4444 4444

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CNF15[1:0]		MODE15[1:0]		CNF14[1:0]		MODE14[1:0]		CNF13[1:0]		MODE13[1:0]		CNF12[1:0]		MODE12[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNF11[1:0]		MODE11[1:0]		CNF10[1:0]		MODE10[1:0]		CNF9[1:0]		MODE9[1:0]		CNF8[1:0]		MODE8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:30, 27:26, 23:22, 19:18, 15:14, 11:10, 7:6, 3:2 **CNFy[1:0]:** Port x configuration bits (y= 8 .. 15)
These bits are written by software to configure the corresponding I/O port.
Refer to *Table 20: Port bit configuration table*.
In input mode (MODE[1:0]=00):
00: Analog mode
01: Floating input (reset state)
10: Input with pull-up / pull-down
11: Reserved
In output mode (MODE[1:0] > 00):
00: General purpose output push-pull
01: General purpose output Open-drain
10: Alternate function output Push-pull
11: Alternate function output Open-drain

Bits 29:28, 25:24, 21:20, 17:16, 13:12, 9:8, 5:4, 1:0 **MODEy[1:0]:** Port x mode bits (y= 8 .. 15)
These bits are written by software to configure the corresponding I/O port.
Refer to *Table 20: Port bit configuration table*.
00: Input mode (reset state)
01: Output mode, max speed 10 MHz.
10: Output mode, max speed 2 MHz.
11: Output mode, max speed 50 MHz.

a) 0x4444 4444 means that each pin is set to 0100, meaning input mode(reset state), floating input(reset state)

(a) Explain the meaning of the Reset Value: 0x4444 4444

(b) To access certain GPIO port's CRL and CRH, can use below method (e.g. for Port A):
GPIOA->CRL
GPIOA->CRH

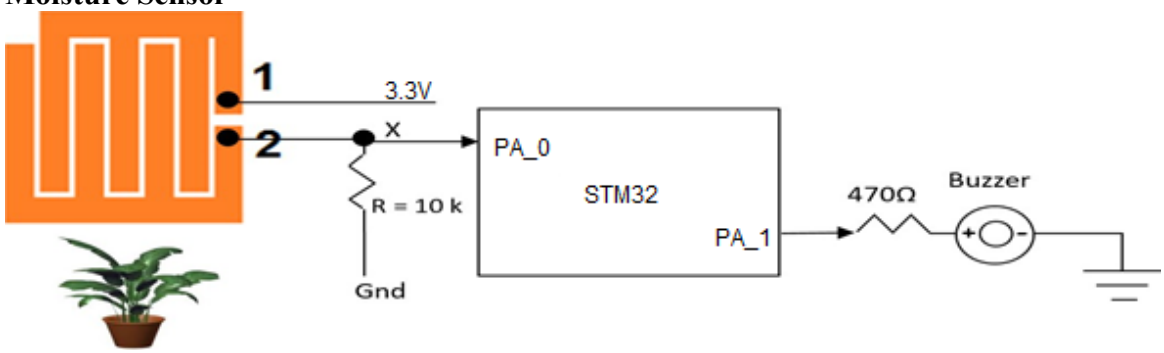
bi)
GPIOB -> CRL &= FFFFFFF0;
GPIOB -> CRL |= 00000004;

bii)
GPIOC -> CRH &= FFFFF0FF
GPIOC -> CRL |= 00000400

Write the codes to:
i) set PB_0 as Output with Push-pull and max output speed of 10MHz.
ii) set PC11 as Output with Push-pull and max output speed of 10MHz.

Q3-2.5 To monitor the soil moisture of a potted plant soil, a moisture sensor and a buzzer are connected to a STM32 microcontroller as shown in the diagram below.

Moisture Sensor



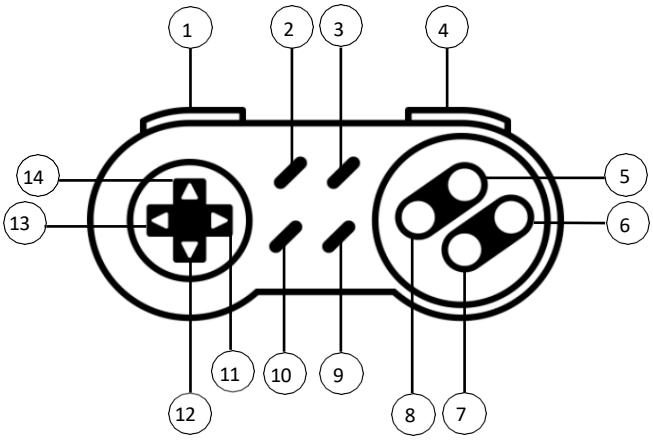
When the soil is moist, the resistance of the sensor will be 0Ω and when the soil is dry, the resistance of the moisture sensor will be very high, moisture sensor can be approximated as an open circuit.,

a. If the soil is dry, what logic value will the STM32 microcontroller read at its PA_0 pin? a) 0, Low
b. To activate the buzzer, what is the logic level required at output pin PA_1? b) 1, High
c. Complete the C code for lines A to G on the next page for the STM32 microcontroller to turn on the buzzer when the soil is dry.

Code	Comments
DigitalIn moistSensorPA0(PA_0); DigitalOut buzzerPA1(PA_1); main() { while(1) {	
if(moistSensorPA0 == 1	// Line A = Check if soil is still moist
{	
buzzerPA1 = 0;	// Line B = Turn off buzzer
}	
else	// Soil is too dry
{	
buzzerPA1 = 1;	// Line C = Turn on the buzzer
thread_sleep_for(2000);	// Line D = Delay for 2 seconds
buzzerPA1 = 0;	// Line E = Turn off the buzzer
thread_sleep_for(2000);	// Line F = Delay for 2 seconds
}	

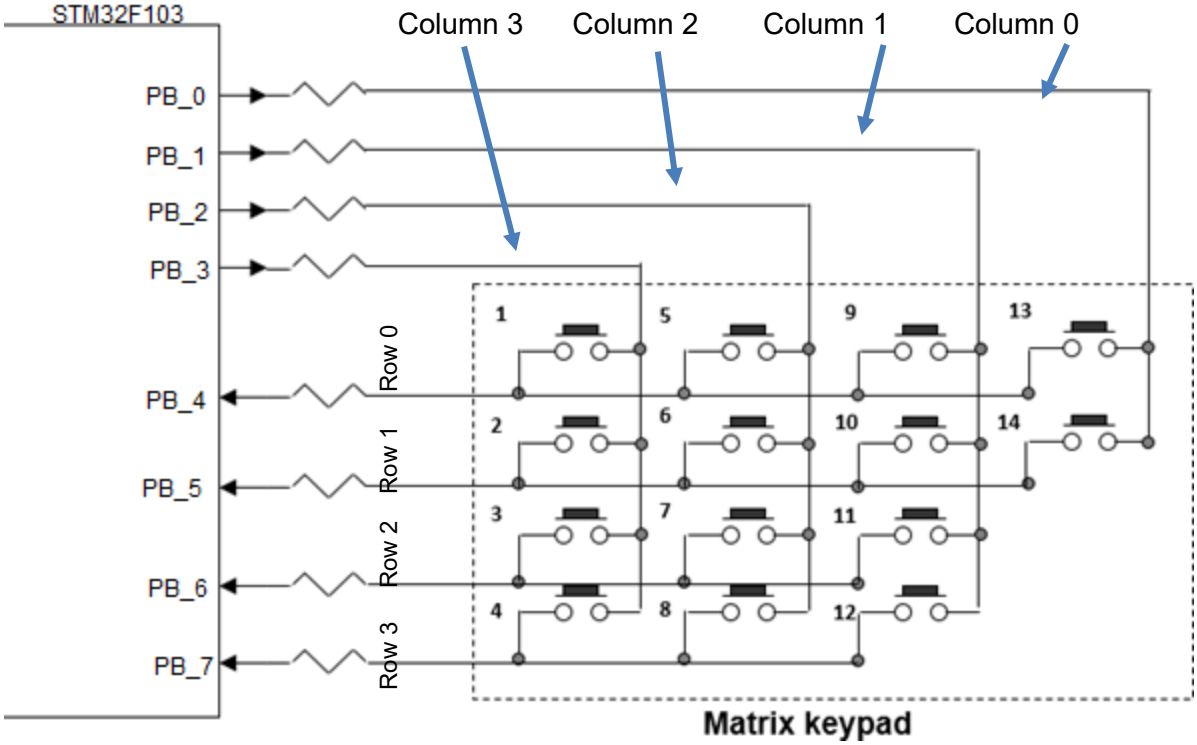
thread_sleep_for(100) } /* End of while(1) loop */	// Line G = Sample moisture sensor every 100 ms

Q3-2.6 A new low cost game controlled is being developed based on the STM32F103 with the following digital input buttons show in the diagram below.



Button	Description
1	Left Alternate
2	Start
3	Stop
4	Right Alternate
5	X
6	A
7	B
8	Y
9	Back
10	Select
11	Right
12	Down
13	Left
14	Up

The buttons on controller gamepad are connected as **a key matrix** in the diagram below. Each keypad column must be enabled for scanning every 10ms.



(a) If the key scanning is based on **Active High inputs**, implement the C/C+ function below to read if any of the game controller direction buttons are pressed (e.g., Left, Right, Up, Down Buttons).

unsigned char GameController_getDirection(void)
{
….
}

Also, you may use below definitions:
PortOut colScanPortB(PortB, 0x0000000F);
BusIn rowScan(PB_4, PB_5, PB_6, PB7);

You also may use the predefined key words: KEY_NONE, KEY_UP, KEY_LEFT, KEY_RIGHT, KEY_DOWN in your codes. For example:

```
#define KEY_NONE                0
#define KEY_LEFT_ALTERNATE      1
#define KEY_START               2
#define KEY_STOP                3
#define KEY_Right_ALTERNATE     4
#define KEY_X                   5
#define KEY_A                   6
#define KEY_B                   7
#define KEY_Y                   8
#define KEY_BACK                9
#define KEY_SELECT              10
#define KEY_RIGHT               11
#define KEY_DOWN                12
#define KEY_LEFT                13
#define KEY_UP                  14
```

```
a) unsigned char GameController_getDirection(void)
{

 unsigned char keyPressed = KEY_NONE;
 colscanPortB.write(0x01);

 if ((rowScan() & 0x01) != 0)
 {
 keyPressed = KEY_LEFT;
 }
 else if ((rowScan() & 0x02) != 0)
 {
 keyPressed = KEY_UP;
 }

 colscanPortB.write(0x02);
 if ((rowScan() & 0x04) != 0)
 {
 keyPressed = KEY_RIGHT;
 }

 else if ((rowScan() & 0x08) != 0)
 {
 keyPressed = KEY_DOWN;
 }

 thread_sleep_for(10);

}
```

(b) The C/C++ function for STM32F103 below decodes any key press. Complete in the missing code to complete the implementation of the key scanning function.

Code	Comments
#undef __ARM_FP #include "mbed.h"	
#define KEY_NONE 0 #define KEY_LEFT_ALTERNATE 1 #define KEY_START 2 #define KEY_STOP 3 #define KEY_Right_ALTERNATE 4 #define KEY_X 5 #define KEY_A 6 #define KEY_B 7 #define KEY_Y 8 #define KEY_BACK 9 #define KEY_SELECT 10 #define KEY_RIGHT 11 #define KEY_DOWN 12 #define KEY_LEFT 13 #define KEY_UP 14	
#define COL_SCAN_DELAY __10__	// Configure column scanning interval
#define COL_MAX ___3__	// Configure number of keypad columns
PortOut colScanPortB(PortB, 0x0000000F);	//Configure Port B inputs and outputs
BusIn rowScan(PB_4, PB_5, PB_6, PB7);	
unsigned char readKey(unsigned char column);	

void main()
{
unsigned char keyPressed = KEY_NONE;	

unsigned char i = 0;	
while (1) {	
for (i=0; i< COL_MAX; i++) {	// Loop over all keypad columns
colScanPortB = 0x01 << i;	// Select key pad column to scan
keyPressed = readKey(i);	// Call the function "readKey()"

/*Display key number on serial monitor*/
if (keyPressed != KEY_NONE)
printf ("Key pressed is: %d\n", keyPressed);
thread_sleep_for(COL_SCAN_DELAY);
}//for
}//while	

}//main	
/* Function to read key*/ unsigned char readKey(unsigned char column) {	

unsigned char keyNumber =
KEY_NONE; keyNumber =rowScan.read();
switch (column)
{
	// Read keypad row inputs
	
	//find out which key is pressed

	
}	
return keyNumber;	// Return the pressed key number
}	

```
case 0:
if ( ( keyNumber & 0x01 ) != 0 )
keyNumber = KEY_LEFT;

else if ( ( keyNumber & 0x02 ) != 0)
keyNumber = KEY_LEFT;

else
keyNumber = KEY_NONE;
break;

case 1:
if ((keyNumber & 0x01) !=0)
keyNumber= KEY_BACK;

else if ((keyNumber & 0x02) !=0)
keyNumber = KEY_SELECT;

else if (keyNumber & 0x04) !=0)
keyNumber = KEY_RIGHT

else if ((keyNumber & 0x08) !=0)
keyNumber = KEY_DOWN

else keyNumber = KEY_NONE;
break;

case 2:
if ((keyNumber & 0x01) !=0)
...
...
```